

Smart Refrigerator Monitoring System

TITLE:

A Smart Refrigerator Monitoring System is an IoTbased project that continuously checks the refrigerator's temperature, door status, and food quantity and alerts the user when something is wrong.

Done By:

PRIYANTH CS -

24MER044

MUTHU KRISHNA P -

24MER033

SRI RAMAN G -

24MEL068

PRADEEP P -

24MER041

Project Overview:

The Smart Refrigerator Monitoring System is an IoT-based project designed to monitor the internal conditions of a refrigerator continuously and provide alerts whenever an abnormal condition is detected. The system uses an ESP32 microcontroller as the main control unit along with different sensors such as a temperature sensor, humidity sensor, magnetic door sensor, and load cell.

In a conventional refrigerator, the user normally has to manually check whether the door is properly closed, whether the temperature is suitable, and whether sufficient food is available. If the refrigerator door is

accidentally left open or the temperature rises, food may spoil and electricity may be wasted. The proposed system addresses these issues through automatic monitoring.

The temperature sensor measures the internal temperature, while the magnetic reed switch detects whether the refrigerator door is open or closed. A load cell can be used to estimate the quantity of selected food items by measuring their weight. The sensor values are processed by the ESP32 and displayed on an OLED display. If the temperature becomes too high or the door remains open for a specified period, the system activates a buzzer.

The ESP32 also provides Wi-Fi connectivity, allowing the refrigerator status to be transmitted to an IoT platform or mobile application. Therefore, the user can monitor the refrigerator remotely.

Problem Statement

Traditional refrigerators are mainly designed to maintain low temperatures and preserve food. However, they generally provide limited information about the actual condition of the stored food and refrigerator operation.

One common problem is that the refrigerator door may accidentally remain open. An open door allows cold air to escape, causing the refrigerator temperature to increase and increasing energy consumption. Another problem is that temperature changes may not be noticed immediately by the user. This can affect the quality and storage life of perishable food.

Users may also forget what food items are available inside the refrigerator or how much of a particular item remains. This can lead to unnecessary purchases and food wastage.

Therefore, a monitoring system is required that can automatically measure important refrigerator parameters and notify the user when abnormal conditions occur.

Proposed Solution

The proposed solution is an ESP32-based Smart Refrigerator Monitoring System.

The ESP32 receives data from different sensors and processes the information according to predefined limits.

Main components

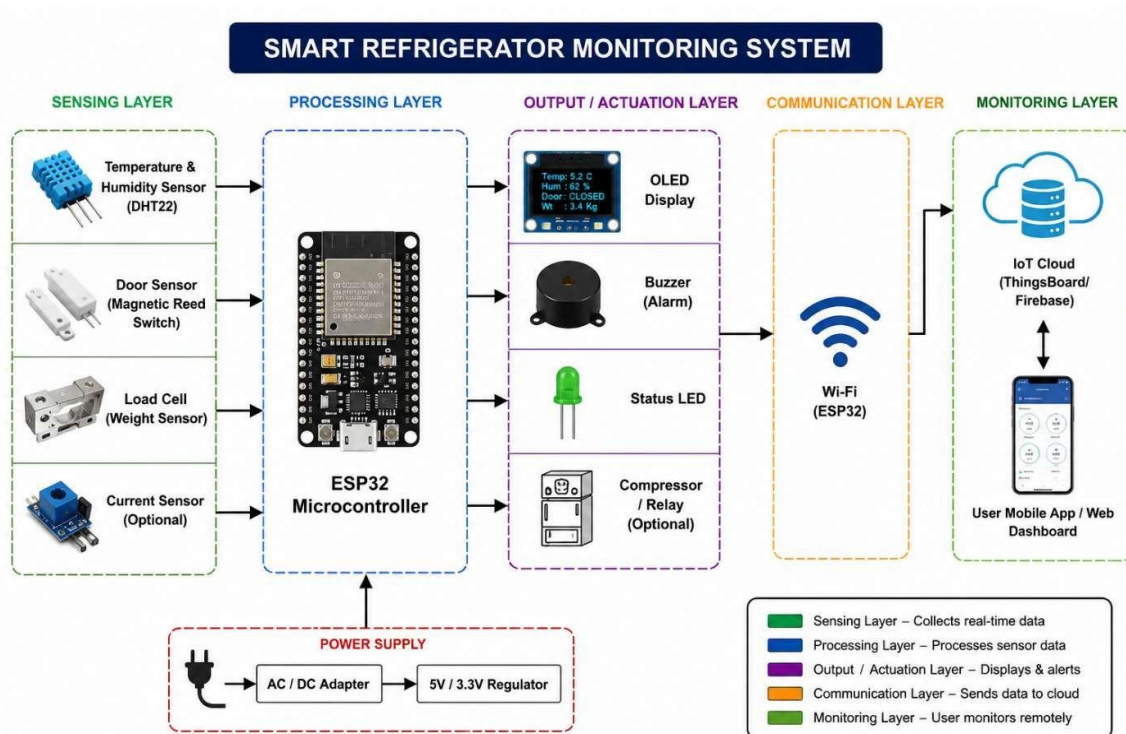
- ESP32 development board
- DHT22 temperature and humidity sensor
- Magnetic reed switch
- Load cell
- HX711 amplifier
- OLED display
- Buzzer
- LED
- Wi-Fi
- Power supply

The temperature sensor is placed inside the refrigerator. The reed switch is installed near the refrigerator door. The load cell is placed below a food-storage tray to measure the approximate weight of selected items.

The ESP32 continuously reads the sensor values. If the temperature rises above the configured limit, the buzzer is activated. If the door remains open for a particular time, a door-open warning is generated.

System Architecture

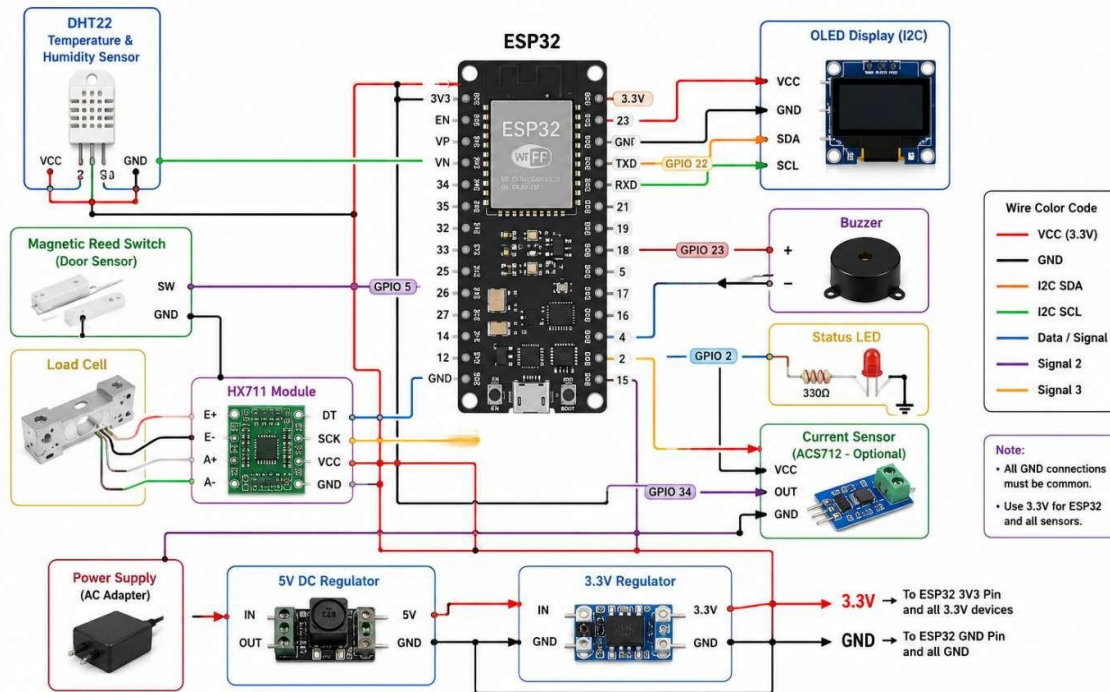
The system consists of four main sections: sensing, processing, communication, and monitoring.



The sensors collect information from the refrigerator. The ESP32 processes this information and decides whether the operating condition is normal or abnormal.

Circuit Design

SMART REFRIGERATOR MONITORING SYSTEM – CIRCUIT DIAGRAM



Coding

The system can be programmed using the Arduino IDE. The following basic program demonstrates temperature, humidity, door and buzzer monitoring.

```
// AIR POLLUTION MONITORING SYSTEM
// ESP8266 NodeMCU + MQ-2 + Buzzer + LED + WiFi
// + ThingSpeak
//
=====
==

#include <ESP8266WiFi.h>
#include <ESP8266HTTPClient.h>

//
=====
==
```

```
// WIFI CONNECTION
```

```
//
```

```
=====
```

```
==
```

```
const char* WIFI_SSID = "praveen";
```

```
const char* WIFI_PASSWORD = "praveen12345";
```

```
//
```

```
=====
```

```
==
```

```
// THINGSPEAK API
```

```
//
```

```
=====
```

```
==
```

```
const char* THINGSPEAK_API_KEY = "0IP4GOX0SMNK3JBV";
```

```
const char* THINGSPEAK_SERVER =
```

```
"http://api.thingspeak.com/update";
```

```
//
```

```
=====
```

```
==
```

```
// MQ-2 Analog Output -> A0
```

```
//
```

```
=====
```

```
==
```

```
#define MQ2_PIN A0
```

```
// Buzzer + -> D6 / GPIO12
```

```
#define BUZZER_PIN D6
```

```
// LED Anode -> D5 / GPIO14 through 220 ohm resistor
```

```
#define LED_PIN D5
```

```
// Gas pollution threshold
```

```
// Adjust this value after testing your MQ-2 sensor
```

```
#define GAS_THRESHOLD 200
```

```
// ThingSpeak upload timer unsigned long
```

```
lastThingSpeakUpdate = 0;
```

```
const unsigned long THINGSPEAK_INTERVAL = 15000;
```

```
//
```

```
=====
```

```
==
```

```
// SETUP
```

```
//
```

```
=====
```

```
==
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  pinMode(MQ2_PIN, INPUT);
```

```
  pinMode(BUZZER_PIN, OUTPUT);
```

```
  pinMode(LED_PIN, OUTPUT);
```

```
  digitalWrite(BUZZER_PIN, LOW);
```

```
  digitalWrite(LED_PIN, LOW);
```

```
//
```

```
=====
```

```
==
```

```

// WIFI CONNECTION
//
=====

Serial.println();
Serial.println("=====");
Serial.println(" AIR POLLUTION MONITORING SYSTEM");
Serial.println("=====");

Serial.println("Connecting to WiFi...");

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}

Serial.println();
Serial.println("WiFi Connected!");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());

//
=====

// ORIGINAL MQ-2 INITIALIZATION
//
=====

```



```

Serial.println("MQ-2 Sensor Initializing...");

delay(2000);

Serial.println("System Ready");
Serial.println("-----");
}

//
=====

==
// LOOP
//
=====

==

void loop()
{
  // Read MQ-2 analog value
  int gasValue = analogRead(MQ2_PIN);

  Serial.print("MQ-2 Gas Level: ");
  Serial.println(gasValue);

  //
=====

==
  // ORIGINAL POLLUTION CHECK
  //
=====

==

```

```

    if (gasValue >= GAS_THRESHOLD)
    {
        digitalWrite(LED_PIN, HIGH);
        digitalWrite(BUZZER_PIN, HIGH);
        Serial.println("WARNING: HIGH GAS LEVEL!");
        Serial.println("Air Quality: POOR");
    }
    else
    {
        digitalWrite(LED_PIN, LOW);
        digitalWrite(BUZZER_PIN, LOW);

        Serial.println("Air Quality: NORMAL");
    }

    Serial.println("-----");

    //
    =====
    ==
    // THINGSPEAK DATA UPLOAD
    //
    =====
    ==

    if (millis() - lastThingSpeakUpdate >= THINGSPEAK_INTERVAL)
    {
        lastThingSpeakUpdate = millis();

        if (WiFi.status() == WL_CONNECTED)
        {
            WiFiClient client;
            HTTPClient http;

```

```
String url = String(THINGSPEAK_SERVER);
url += "?api_key="; url +=
THINGSPEAK_API_KEY;
url += "&field1=";
url += String(gasValue);
Serial.println("Sending data
to ThingSpeak...");

http.begin(client, url);

int httpCode = http.GET();

if (httpCode > 0)
{
    Serial.print("ThingSpeak Response: ");
    Serial.println(httpCode);

    if (httpCode == 200)
    {
        Serial.println("ThingSpeak Update Successful!");
    }
else
    {
        Serial.println("ThingSpeak Update Failed!");
    }
}
else
{
    Serial.print("ThingSpeak Error: ");
    Serial.println(http.errorToString(httpCode));
}

http.end();
```

```
    }  
else  
    {  
        Serial.println("WiFi Disconnected!");  
    }  
  
    Serial.println("-----");  
}  
  
delay(1000);  
}
```

System Working

When the system is switched ON, the ESP32 initializes all the sensors and establishes Wi-Fi connectivity.

The temperature sensor continuously measures the internal refrigerator temperature. The humidity sensor measures the moisture level. The magnetic reed switch checks whether the refrigerator door is open or closed.

The load cell measures the weight of food items placed on the monitoring platform.

The ESP32 compares the sensor values with predefined limits.

For example, if the refrigerator temperature becomes higher than the selected threshold:

Similarly, when the refrigerator door is opened, the ESP32 starts monitoring the duration. If the door remains open beyond the configured time, the buzzer is activated.

The current refrigerator status is shown on the OLED display and can also be transmitted to a mobile or web dashboard through Wi-Fi.

Bill of Materials

S.No.	Component	Quantity	Approx. Cost
1	ESP32	1	₹400–₹600
2	DHT22	1	₹150–₹300
3	Magnetic Reed Switch	1	₹20–₹50
4	Load Cell	1	₹150–₹250
5	HX711 Module	1	₹50–₹100
6	OLED Display	1	₹100–₹200
7	Buzzer	1	₹10–₹30
8	LED	2	₹10–₹20
9	Breadboard	1	₹80–₹150
10	Jumper Wires	1 set	₹50–₹100
11	Power Supply	1	₹100–₹200
12	Prototype Body	1	₹200–₹500

Estimated Total

₹1,320–₹2,500

The actual cost varies according to the components and supplier.

Prototype Implementation

The prototype can be constructed using a small insulated box representing the refrigerator.

First, the ESP32 is placed outside the refrigerator model. The temperature sensor is positioned inside the storage compartment. The magnetic reed switch is installed near the door so that the ESP32 can detect door movement.

A small food tray is placed on the load cell. The load cell is connected to the HX711 module, which is then connected to the ESP32.

The OLED display and buzzer are placed outside the refrigerator so that the user can easily see the status and hear warning alerts.

Implementation Steps

1. Construct the refrigerator prototype.
2. Install the temperature sensor.
3. Install the magnetic reed switch.
4. Install the load cell and food tray.
5. Connect the HX711 module.
6. Connect OLED and buzzer.
7. Connect all components to the ESP32.
8. Upload the program.
9. Calibrate the load cell.
10. Test different refrigerator conditions.
11. Connect the ESP32 to Wi-Fi.
12. Monitor the system remotely.

Results & Performance Analysis

The developed prototype is expected to continuously monitor the refrigerator environment and detect abnormal conditions.

Performance Table

Test Condition	Expected Result
Normal temperature	System operates normally
High temperature	Buzzer activated

Door closed	CLOSED displayed
Door opened	OPEN detected
Door open for long time	Warning generated
Test Condition	Expected Result
Food quantity changed	Weight reading changes
Normal Wi-Fi	Data transmitted
Wi-Fi unavailable	Local monitoring continues
Sensor failure	Error condition displayed
Expected Results	

The system successfully demonstrates the concept of an intelligent refrigerator monitoring unit. Temperature and humidity can be monitored continuously, while the door sensor provides immediate information about the door condition.

The load cell provides an approximate measurement of food quantity for the monitored tray. The OLED provides local information, while the buzzer provides an immediate warning during abnormal conditions.

The Wi-Fi capability of the ESP32 makes it possible to extend the project into a complete IoT system where refrigerator data can be monitored remotely.

Advantages

- Real-time monitoring.
- Automatic warning system.
- Reduced possibility of food spoilage.
- Door-open detection.
- Food quantity monitoring.

- Low-cost implementation.
- Wi-Fi-based remote monitoring.
- Expandable to AI and cloud technologies.

Conclusion

The Smart Refrigerator Monitoring System provides an effective and low-cost approach for converting a conventional refrigerator into a smart monitoring system. By combining sensors, ESP32, OLED display, buzzer and IoT communication, the system can monitor important refrigerator parameters and notify users when abnormal conditions occur.

The project can be further enhanced by adding food expiry reminders, camera-based food recognition, cloud data logging, energyconsumption analysis, and predictive maintenance, making it suitable for an advanced IoT or final-year engineering project.